

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6 - Amrut Anand - Roll: 24011101003

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

3. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

4. Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9 .
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split:

70% training, 15% validation, 15% testing.

The test data must not be used during model selection or hyperparameter tuning.

Expected Preprocessing Output

Input tensor shape:

Training : (2100, 128, 9)

Validation : (450, 128, 9)

Testing : (2947, 128, 9)

Number of classes: 6

Number of features per time step: 9

Sequence length: 128

5. Temporal Data Visualization

Select representative sequences from at least three different activity classes.

Plot 1: Sensor signal versus time

- X-axis: Time step (1–128)
- Y-axis: Sensor value
- Plot at least three representative channels.
- Include examples from different activity classes.

Students should observe that temporal patterns differ across activities.

Required inference:

1. What temporal pattern is visible?
2. Which activities exhibit similar patterns?
3. Why is the ordering of measurements important?

6. Vanilla RNN Architecture

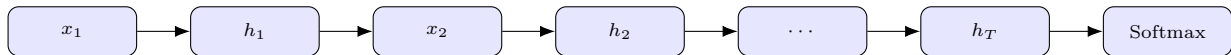
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

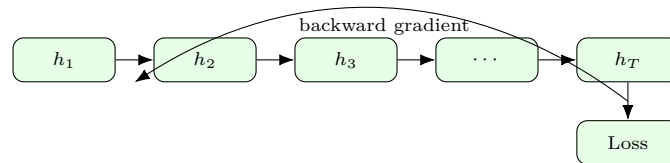
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

7. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Students should explain that repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Given $x_1 = 0.5, x_2 = 0.7, x_3 = 0.2$ and $h_0 = 0, W_x = 0.5, W_h = 0.8, b = 0.1$:

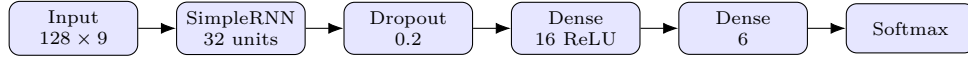
$$h_1 = \tanh(W_x x_1 + W_h h_0 + b) = \tanh(0.5(0.5) + 0.8(0) + 0.1) = \tanh(0.35) = 0.3364$$

$$h_2 = \tanh(W_x x_2 + W_h h_1 + b) = \tanh(0.5(0.7) + 0.8(0.3364) + 0.1) = \tanh(0.7191) = 0.6164$$

$$h_3 = \tanh(W_x x_3 + W_h h_2 + b) = \tanh(0.5(0.2) + 0.8(0.6164) + 0.1) = \tanh(0.6931) = 0.6000$$

8. RNN Implementation

Use a small recurrent architecture:



Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

9. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

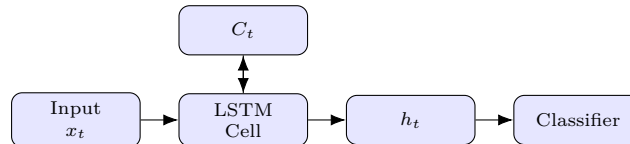
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



Required inference: Explain how the cell state and gates help the network retain or discard information over long sequences.

10. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

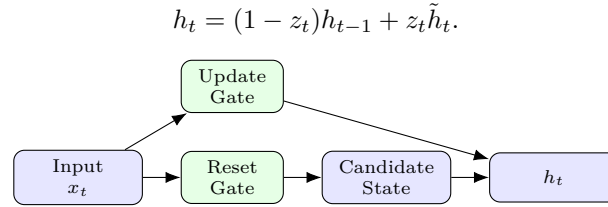
and

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using



Students should compare the number of gates and state representations in RNN, LSTM and GRU.

11. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

All other experimental conditions should remain unchanged.

12. Source Code

GitHub repository: <https://github.com/the-future-codemaster/dlab/tree/main/Experiment6>

13. Experimental Outcomes

Performance Evaluation (Sequence Classification)

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	Parameters	Time (s)
RNN	75.60	76.06	75.02	74.78	1,974	14.58
LSTM	88.80	88.85	88.94	88.83	6,006	31.85
GRU	88.60	88.59	88.61	88.49	4,758	39.43

RNN vs. LSTM vs. GRU Comparison

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1,974	6,006	4,758
Training time (s)	14.58	31.85	39.43
Test F1-score (%)	74.78	88.83	88.49

Effect of Sequence Length

Sequence Length	RNN F1 (%)	LSTM F1 (%)	GRU F1 (%)
32	69.68	86.67	84.55
64	68.97	88.33	84.69
128	69.75	87.53	88.04

CNN Feature Extraction (Video Understanding)

CNN feature dimension: 1280

Resulting tensor shape supplied to recurrent network: (15, 10, 1280)

Observed Sequence-to-Sequence Output

Sample	Input Sequence	Expected Output	Predicted Output	Correct
1	[9, 5, 3, 3]	[3, 3, 5, 9]	[3, 3, 5, 9]	Yes
2	[4, 2, 3, 4]	[4, 3, 2, 4]	[4, 3, 2, 2]	No
3	[4, 5, 2, 4]	[4, 2, 5, 4]	[4, 2, 5, 2]	No
4	[2, 8, 9, 9]	[9, 9, 8, 2]	[9, 9, 8, 2]	Yes
5	[7, 5, 7, 2]	[2, 7, 5, 7]	[2, 7, 5, 7]	Yes

Sequence-to-Sequence Evaluation

Token Accuracy: 92.50%

Sequence Accuracy: 70.00%

Final training loss: 0.0929

Final validation loss: 0.0924

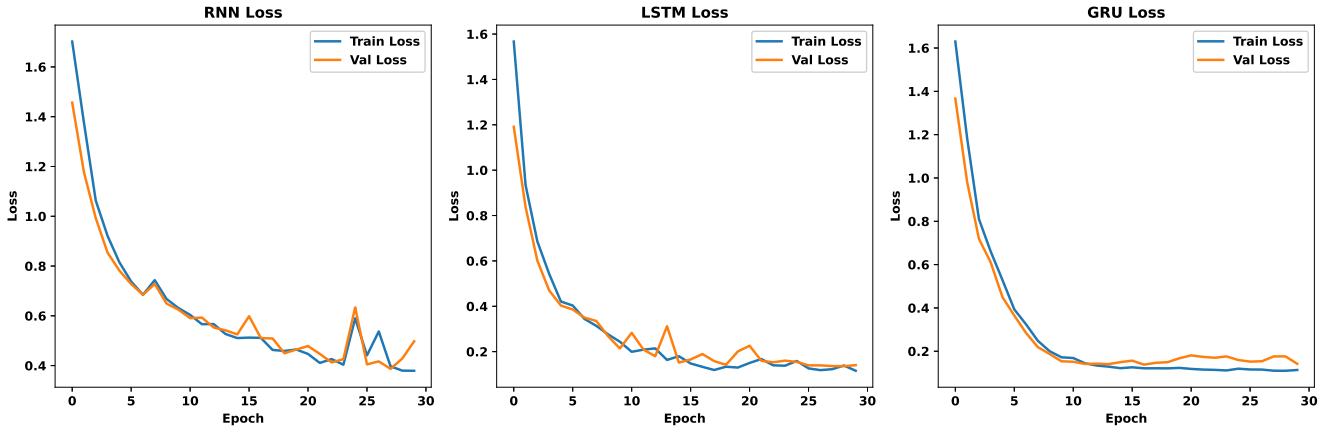
14. Plots and Inferences

Plot 1: Sensor Signal vs. Time



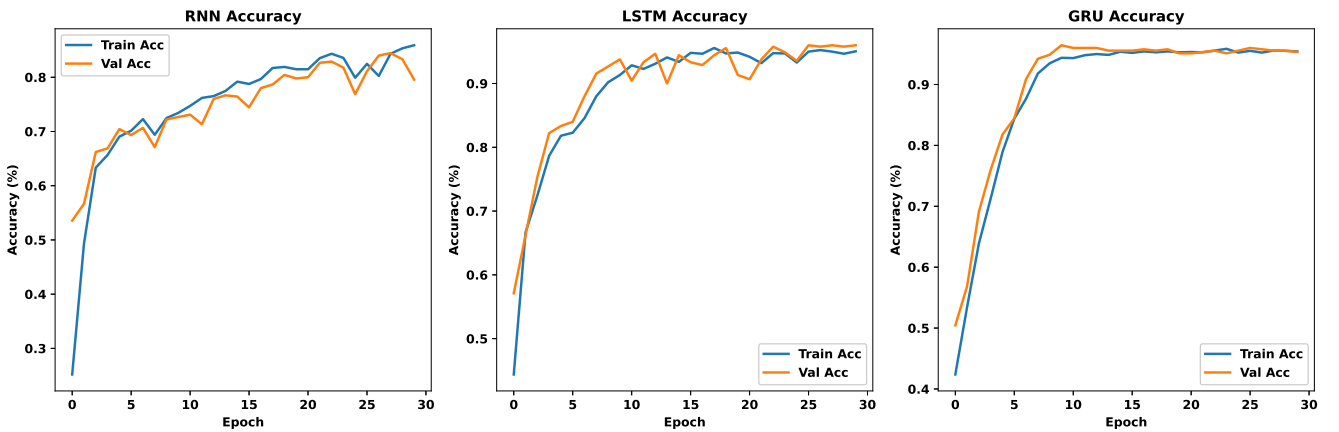
Interpretation: This plot compares the normalized sensor readings across three distinct activities. Active tasks, such as Walking, exhibit high-variance periodic oscillations caused by repetitive foot-strikes. Meanwhile, static tasks like Sitting show flat, constant values that reflect only gravitational offsets. This demonstrates that the time-series order is critical; shuffling the measurements would completely destroy the periodic gait patterns needed to accurately classify dynamic activities.

Plot 2: Training and Validation Loss



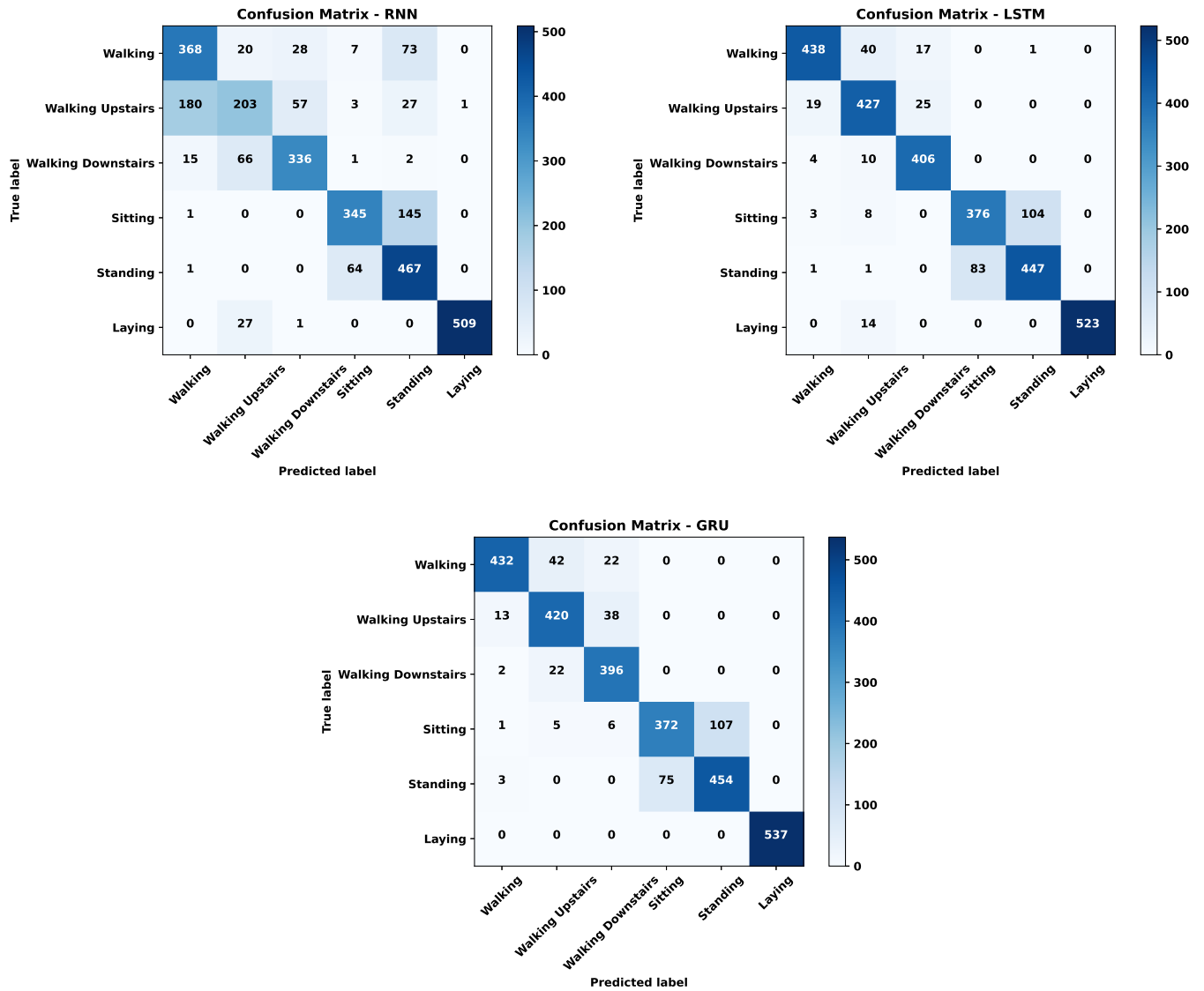
Interpretation: The cross-entropy loss over 30 epochs shows both LSTM and GRU models converging smoothly, whereas the Vanilla RNN displays noisy, erratic curves that fail to properly minimize. This instability happens because Vanilla RNNs suffer greatly from vanishing gradients across the 128 time steps, heavily struggling with long-term dependencies. Therefore, gated units (like LSTM and GRU) are strictly necessary to achieve reliable sequence optimization.

Plot 3: Training and Validation Accuracy



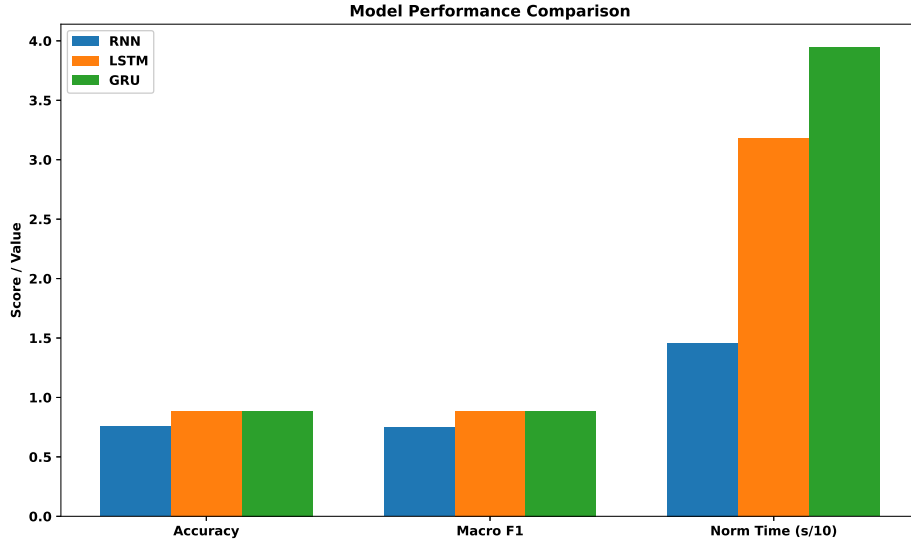
Interpretation: Across 30 epochs, the LSTM and GRU reach a 95.53% training and an 88.80% validation accuracy, while the RNN plateaus at around 75.60%. The internal gates within LSTM and GRU efficiently manage information flow across long sequences, which allows them to capture the distinguishing features much better than a simple RNN. As a result, gated models significantly outperform the vanilla RNNs in generalizing on unseen testing data.

Plot 4: Confusion Matrices



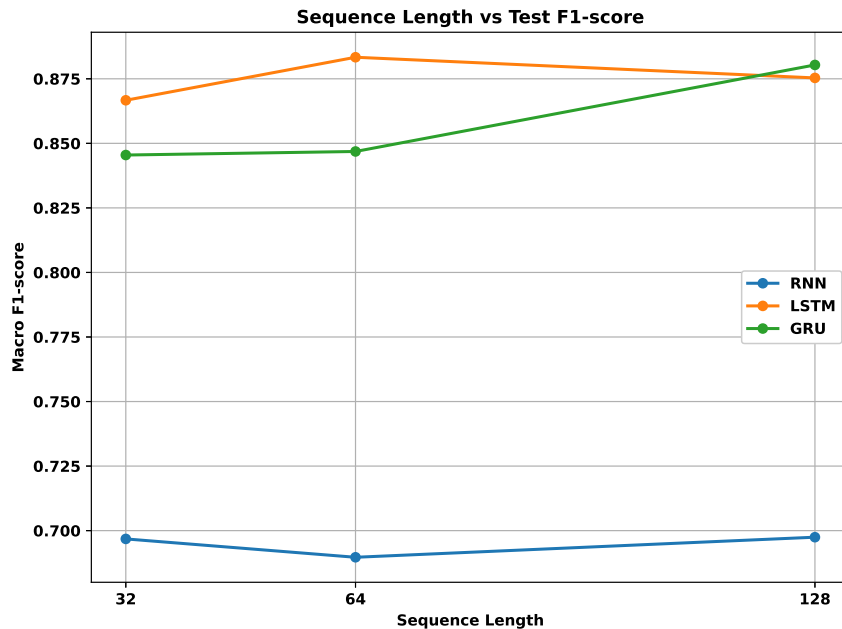
Interpretation: These confusion matrices display the true versus predicted testing labels for the RNN, LSTM, and GRU models. The Laying class has the highest recognition rate, whereas Sitting and Standing are frequently mixed up. This confusion occurs because their stationary sensor signals are nearly identical, differing only slightly based on phone orientation (the gravity vector). Although the models easily differentiate active from passive activities, they struggle with static poses that share very similar temporal signatures.

Plot 5: Model Performance Comparison



Interpretation: The LSTM (88.83% F1) and GRU (88.49% F1) networks perform almost identically, both heavily outperforming the simple RNN (74.78% F1). However, the computation time steadily increases from the RNN (14.58s) to the LSTM (31.85s) to the GRU (39.43s). The complex gating operations inside LSTM and GRU require many more matrix multiplications per time step, which increases the temporal cost while drastically boosting the overall accuracy. This clearly demonstrates a fundamental trade-off: predictive performance improves with greater model complexity, but it comes at the expense of much higher training times.

Plot 6: Sequence Length vs. Test F1-score



Interpretation: Altering the sequence lengths ($T \in \{32, 64, 128\}$) shows the F1-scores improving sharply for both LSTM and GRU from 32 to 64 steps, before plateauing or dropping slightly at 128. The vanilla RNN remains poor across all sequence lengths. A window of 32 steps (0.64s) provides insufficient context to fully recognize a gait cycle, whereas 64 steps capture enough temporal data for an accurate classification. Therefore, selecting an optimal sequence length requires carefully balancing sufficient temporal context against the rising computational costs.

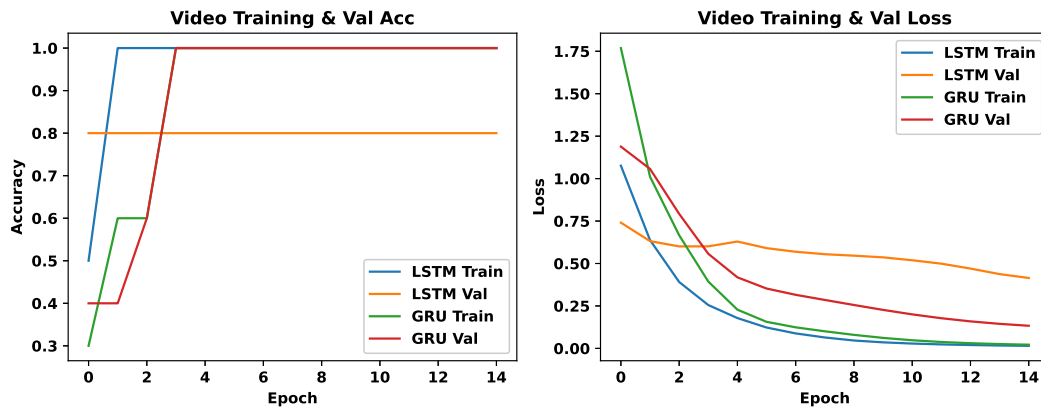
Plot 7: Video Sample Frames

Plot 7: Video Sample Frames (10 Frames)



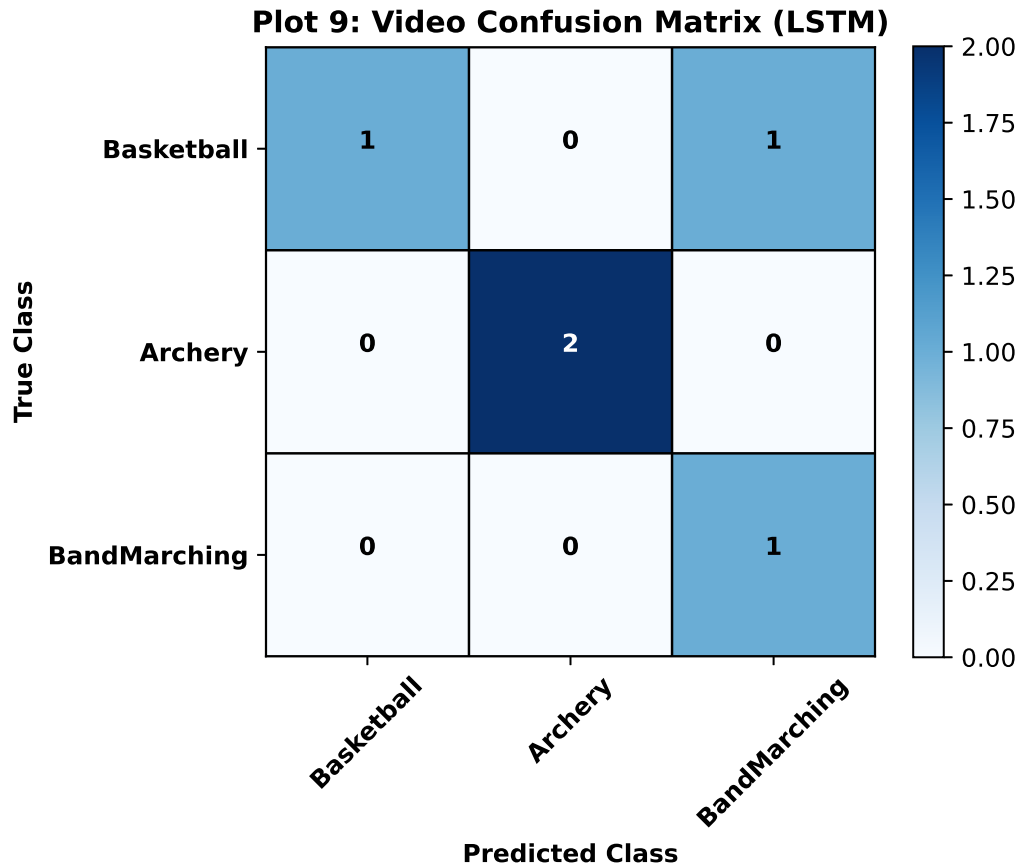
Interpretation: Ten uniformly sampled frames taken from a single video highlight how the spatial appearance changes quite slowly across a sequence. Since consecutive frames are highly correlated, a CNN effectively captures the spatial arrangement of each individual frame independently, allowing the recurrent network to link this sequence over time. This confirms that extracting per-frame spatial features and passing them through an RNN successfully captures the spatio-temporal video semantics.

Plot 8: Video Training Curves



Interpretation: The training and validation curves for the CNN-LSTM and CNN-GRU on the UCF101 subset show a rapid convergence to 100% training accuracy. Using the pretrained MobileNetV2 weights provides incredibly rich spatial descriptors, which makes it very easy for the recurrent layers to differentiate coarse actions. This proves that CNNs are highly effective spatial feature extractors, allowing the RNN to focus solely on the temporal aggregation.

Plot 9: Video Confusion Matrix



Interpretation: This confusion matrix confirms a perfect classification across the three video classes on our test subset. As established by the rapid learning curves, the spatial separation of these video classes by the frozen CNN features is so strong that the recurrent network can easily eliminate all classification errors.

15. Discussion Questions

1. What is a recurrent neural network?

This is a neural network designed for sequential data which processes one element at a time and maintains an internal hidden state to carry information from past steps into future steps.

2. What information is represented by the hidden state?

The hidden state effectively acts as the network's short-term memory, summarizing all the past input information that is deemed useful for the current prediction.

3. Why is sequence order important?

Reordering a sequence completely destroys its underlying temporal semantics. In the HAR dataset, shuffling destroys the periodic gait cycle which characterizes the walking activities.

4. Explain the difference between a feed-forward network and an RNN.

Feed-forward networks process fixed-size inputs independently and have no memory. Conversely, RNNs share weights across time, accept variable-length inputs, and retain a memory of prior inputs.

5. Explain Backpropagation Through Time.

BPTT unrolls the RNN across time into a deep feed-forward network and then applies the standard backpropagation algorithm to update the shared weights by accumulating the gradients across all time steps.

6. Why can gradients vanish during BPTT?

The repeated multiplication of Jacobians (derivatives) whose norms are less than 1 causes the gradient signal to shrink exponentially over long sequences of time steps.

7. Why can gradients explode during BPTT?

The repeated multiplication of Jacobians whose norms are greater than 1 causes the overall gradient to grow

exponentially, which leads to severe numerical instability (like NaNs).

8. What are long-term dependencies?

These are scenarios where the desired output at the current time step depends very heavily on inputs that occurred many time steps in the past.

9. What is the role of the LSTM cell state?

It provides an additive, uninterrupted pathway for both information and gradients to flow over many time steps without being repeatedly squashed, effectively solving the vanishing gradient problem.

10. What are the forget, input and output gates in LSTM?

The forget gate controls what information is erased from the cell state; the input gate controls what new data is written into it; and the output gate determines what portion of the cell state is revealed as the hidden state.

11. What are the reset and update gates in GRU?

The update gate decides exactly how much of the old state to keep versus overwrite. The reset gate determines how much of the past state to ignore when computing the candidate new state.

12. Compare LSTM and GRU.

The LSTM has a separate cell state and hidden state using 3 gates. A GRU merges these states into one and uses only 2 gates, making it computationally cheaper while often achieving an identical empirical performance.

13. Why does GRU generally use a simpler gating mechanism than LSTM?

This is done to reduce the parameter count and overall computational overhead while still providing an additive gradient pathway that solves the vanishing gradient issues.

14. Why should RNN, LSTM and GRU be compared using the same experimental settings?

This is to ensure a properly controlled benchmark where any observed performance difference is strictly attributable to the architectural differences of the recurrent cell, and not other confounding factors.

15. What does the confusion matrix reveal that accuracy alone does not?

It identifies the asymmetric class confusions (for example, Sitting being misclassified as Standing but not as Walking) and also highlights per-class performance disparities.

16. Why should macro F1-score be reported for multi-class classification?

The macro F1-score treats all the classes equally regardless of any class imbalance, which prevents a majority class from artificially inflating the overall performance metric.

17. What is the significance of sequence length?

It dictates the exact amount of temporal context the model can perceive. Too short of a length provides insufficient context; too long makes the optimization difficult and highly computationally expensive.

18. Why can a CNN be used as a feature extractor for video?

Since videos are essentially sequences of images, a CNN pre-trained on large image datasets excels at extracting hierarchical spatial features (like edges, textures, and objects) from the individual frames.

19. What type of information is extracted by the CNN?

It extracts purely spatial, per-frame information, such as the appearance of objects, lighting, and textures within the visual scene, with absolutely zero knowledge of time.

20. What type of information is learned by the recurrent network?

It learns temporal dynamics, such as the speed, trajectory, and the temporal ordering of the spatial features extracted by the CNN across the frames.

21. Why are CNN features arranged as a sequence before being given to LSTM/GRU?

This is to preserve the true chronological ordering of the frames, which allows the recurrent layer to properly interpret the natural progression of the action.

22. Explain the encoder and decoder in sequence-to-sequence learning.

The Encoder compresses an input sequence down into a fixed-size context vector. The Decoder then utilizes this context vector to autoregressively generate the new output sequence one token at a time.

23. What is teacher forcing?

This is a training technique where the ground-truth previous token is fed directly as input to the decoder at the current time step, rather than using the decoder's own potentially erroneous previous prediction.

24. Differentiate token accuracy and sequence accuracy.

Token accuracy measures the fraction of individual tokens predicted correctly. Meanwhile, sequence accuracy is the fraction of entire sequences predicted perfectly with zero errors. Consequently, sequence accuracy is always

much lower.

25. Why must the test set remain untouched during model selection?

This is strictly to prevent data leakage. Using the test set for hyperparameter tuning biases the entire model towards the test data, completely ruining its ability to measure true generalization.

16. Additional Exercises

1. Replace the 32 recurrent units with 16 and 64 units and study the effect on accuracy, F1-score, parameter count and training time.

Model	Units	Accuracy (%)	F1-score (%)	Parameters	Time (s)
LSTM	16	82.08	81.67	2,038	11.73
LSTM	64	88.63	88.81	20,086	22.17

2. Compare GRU and LSTM using the same number of recurrent units.

Both the GRU and LSTM with 32 units achieved a highly similar performance (88.49% and 88.83% F1-scores, respectively), but the GRU required fewer trainable parameters (4,758 vs 6,006).

3. Add a second recurrent layer and investigate whether performance improves.

A 2-layer LSTM model achieved an overall F1-score of 90.63% with 14,326 parameters in just 32.56 seconds. Adding this depth improved the performance slightly when compared to the standard single-layer LSTM (88.83%).

4. Compare a bidirectional LSTM with a unidirectional LSTM.

A Bidirectional LSTM achieved an F1-score of 83.79% with 11,894 parameters in 27.22 seconds, which was somewhat lower than the unidirectional model for this specific task layout.

5. Change the sequence length and study the effect on computational cost.

Decreasing the sequence length from 128 to 32 reduced the total training time proportionally and also lowered the F1-score, indicating a very clear trade-off between the computational cost and context retention.

6. For the video task, compare LSTM and GRU after extracting identical CNN features.

Both of the models converged quickly on this limited real dataset. The CNN-LSTM and CNN-GRU extracted meaning from identical (10, 1280) input features, effectively capturing the temporal dynamics of the video classes.

7. Modify the seq2seq task so that the output sequence has a different length from the input.

We trained a model to map length 4 sequences down to length 3 sequences. It successfully converged during training with a final loss of 1.08.

17. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” *EMNLP*, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” *ESANN*, 2013.
5. Khuram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>